Online Gym Management System

A Minor Project Report

Submitted in partial fulfilment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

By

Tanvi Patil

EN23CS3011077

Under the Guidance of

Prof. Rashmi Vijaywargiya, Prof. Arjun Dixit

Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

APRIL-2026

Report Approval

The project work "Online Gym Management System" is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn there in; but approve the “Project Report” only for the purpose for which it has been submitted.

Internal Examiner:

Name: _____

Designation: _____

External Examiner:

Name: _____

Designation: _____

Declaration

We hereby declare that the project entitled "Online Gym Management System" submitted in partial fulfilment for the award of the degree of Bachelor of Technology in Computer Science and Engineering, completed under the supervision of Prof. Rashmi Vijaywargiya, Prof. Arjun Dixit, Department of Computer Science and Engineering, Faculty of Engineering, Medicaps University Indore, is an authentic work.

Further, we declare that the content of this project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the student(s) with date

Certificate

We certify that the project entitled “Online Gym Management System” submitted in partial fulfilment for the award of the degree of Bachelor of Technology by Tanvi Patil is the record carried out by her under our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Prof. Rashmi Vijaywargiya
Arjun Dixit
Medicaps University, Indore

Prof. (Dr.) Kailash Chandra Bandhu
Head of the Department
Computer Science & Engineering
Medicaps University, Indore

Acknowledgement

We would like to express our deepest gratitude to the Honorable Chancellor, Shri R C Mittal, who has provided every facility to successfully carry out this project, and our profound indebtedness to Prof. (Dr.) D. K. Patnaik, Vice Chancellor, Medi-Caps University, whose unfailing support and enthusiasm has always boosted our morale.

We also thank Prof. Ratnesh Litoriya, Dean, Faculty of Engineering, Medicaps University, for giving us the opportunity to work on this project. We would also like to thank our Head of the Department Prof. (Dr.) Kailash Chandra Bandhu for his continuous encouragement for the betterment of the project.

Special thanks to our project guides Prof. Rashmi Vijaywargiya and Prof. Arjun Dixit for their invaluable guidance, technical insights, and constant support throughout the development of Online Gym Management System.

Tanvi Patil

B.Tech. III Year

Department of Computer Science & Engineering

Faculty of Engineering

Medicaps University, Indore

Abstract

The Online Gym Management System is a web-based application designed to automate and streamline the daily operations of a gym using modern web technologies. The system addresses common challenges faced by fitness centers, such as manual record-keeping, inefficient member management, difficulty in tracking payments, and lack of centralized data access. By digitizing these processes, the system improves accuracy, efficiency, and overall user experience.

The application is developed using the Django framework in Python, following the Model-View-Template (MVT) architecture, and utilizes a relational database (SQLite/MySQL) for secure data storage. The system provides role-based access for three types of users: Admin, Trainer, and Member. The Admin can manage members, trainers, membership plans, and payments through a centralized dashboard. Trainers can view assigned members and monitor their progress, while members can register, log in, view their subscription details, and track payment history.

The web interface is built using HTML, CSS, and Bootstrap, ensuring a responsive and user-friendly design compatible with both desktop and mobile devices. The system includes modules such as user authentication, member and trainer management, plan creation, and payment tracking. By reducing paperwork and minimizing human errors, the Online Gym Management System enhances operational efficiency and provides a scalable solution for modern gym management.

Keywords:

•Online Gym Management System
• Django Framework
• Python Web Development
• Role-Based Access Control
• Member & Trainer Management
• Payment Tracking System
• Database Management (SQLite/MySQL)
• Web Application Development
• Authentication System
• Responsive UI (Bootstrap)

Table of Contents

List of Figures

List of Tables

Abbreviations

1. GMS – Gym Management System
2. UI — User Interface
3. GUI — Graphical User Interface
4. HTML — HyperText Markup Language
5. CSS — Cascading Style Sheets
6. JS — JavaScript
7. HTTP — HyperText Transfer Protocol
8. DB — Database
9. SQL — Structured Query Language
10. ORM — Object Relational Mapper
11. API — Application Programming Interface
12. MVT — Model View Template (Django Architecture)
13. IDE — Integrated Development Environment
14. CPU — Central Processing Unit
15. RAM — Random Access Memory
16. OS — Operating System
17. URL — Uniform Resource Locator
18. CRUD — Create, Read, Update, Delete
19. RBAC — Role-Based Access Control
20. UX — User Experience

Chapter 1: Introduction

1.1 Introduction

The rapid growth of the fitness industry has increased the need for efficient and organized management systems in gyms and fitness centers. Many gyms still rely on manual methods such as registers or spreadsheets to manage member records, trainer assignments, payment details, and membership plans. These traditional approaches are time-consuming, prone to errors, and inefficient, especially as the number of members increases. Issues such as incorrect data entry, difficulty in tracking membership expiry, and lack of centralized access often affect the overall management and user experience.

The **Online Gym Management System** addresses these challenges by providing a centralized, automated, and user-friendly web-based solution. The system allows users to perform various operations such as member registration, trainer management, plan creation, and payment tracking efficiently. It ensures accurate data storage, quick access to information, and reduced administrative workload.

The system is developed using Python with the Django framework, following the Model-View-Template (MVT) architecture. It uses a relational database such as SQLite or MySQL to securely store all data related to members, trainers, plans, and payments. The application provides role-based access for Admin, Trainer, and Member, ensuring proper control and security of data.

The web interface is designed using HTML, CSS, and Bootstrap, making it responsive and accessible on both desktop and mobile devices. By digitizing gym operations, the Online Gym Management System enhances efficiency, reduces errors, and provides a scalable solution for modern fitness centers.

1.2 Literature Review

1. **Web-Based Management Systems**

Recent studies (2022–2025) highlight the growing adoption of web-based management systems in various domains, including healthcare, education, and fitness.

These systems provide centralized data storage, real-time access, and improved efficiency compared to manual methods. Research published in IEEE and Springer journals shows that web applications significantly reduce administrative workload and improve data accuracy. The Online Gym Management System follows this approach by providing a centralized platform for managing members, trainers, and payments.

2. **Django Framework for Web Development**

Research indicates that the Django framework is widely used for rapid and secure web application development. Its Model-View-Template (MVT) architecture, built-in authentication system, and Object Relational Mapper (ORM) make it suitable for developing scalable management systems. Studies (2023–2025) show that Django-based applications provide better security against common threats such as SQL injection and cross-site scripting, making it ideal for handling sensitive user data in gym management.

3. **Database Management Systems**

Efficient data management is crucial for any management system. Research shows that relational databases such as SQLite and MySQL are widely used due to their reliability, structured data storage, and support for complex queries. These databases help maintain data integrity and reduce redundancy. In the proposed system, a relational database is used to store information related to members, trainers, plans, and payments securely.

4. **Role-Based Access Control (RBAC)**

Role-Based Access Control is an important concept in modern web applications. Studies (2023–2024) emphasize that RBAC improves system security by restricting access based on user roles such as Admin, Trainer, and Member. This ensures that sensitive operations are only performed by authorized users. The Online Gym Management System implements RBAC to provide secure and controlled access to different functionalities..

5. **User Interface and User Experience (UI/UX)**

Research highlights that a well-designed user interface improves usability and user satisfaction. Modern web applications use responsive design frameworks like Bootstrap to ensure compatibility across devices. Studies show that clean layouts, intuitive navigation, and mobile responsiveness enhance user engagement. The proposed system adopts a responsive and user-friendly interface to provide a smooth experience for all users.

1.3 Objectives

1. Develop a web-based Online Gym Management System using Python and the Django framework

2. Automate member registration and maintain accurate records of members and trainers

3. Implement role-based access control for Admin, Trainer, and Member users

4. Create and manage membership plans with details such as price and duration

5. Develop a payment management system to track member payments and membership validity

6. Provide a user-friendly and responsive interface accessible on desktop and mobile devices

7. Store and manage all system data securely using a relational database (SQLite/MySQL)

8 Improve overall efficiency by reducing manual work and minimizing human errors

1.4 Significance

The Online Gym Management System has practical significance across multiple areas:

Gym Administration:
Helps gym owners and administrators efficiently manage member records, trainer assignments, membership plans, and payments, reducing manual work and improving accuracy.

User Convenience:
Provides members with easy access to their profiles, subscription details, and payment history, enhancing user experience and transparency.

Operational Efficiency:
Automates routine tasks such as registration, payment tracking, and record maintenance, saving time and minimizing human errors.

Academic Contribution:
Demonstrates the practical implementation of web development concepts using Python and the Django framework, including database management, authentication, and role-based access control in a real-world application.

1.5 Research Design

Phase 1 — Requirement Analysis
The system requirements are identified by analyzing the needs of gym management, including member registration, trainer assignment, plan management, and payment tracking. User roles (Admin, Trainer, Member) and their functionalities are clearly defined.

Phase 2 — System Design
The system architecture is designed using the Model-View-Template (MVT) pattern of Django. Database schema is created to manage entities such as Members, Trainers, Plans, and Payments. ER diagrams and data flow diagrams are prepared to visualize system structure.

Phase 3 — Database Implementation
A relational database (SQLite/MySQL) is implemented to store all system data securely. Tables are created for users, members, trainers, plans, and payments, ensuring proper relationships and data integrity.

Phase 4 — Backend Development
The backend is developed using Python and Django. Models, views, and URLs are implemented to handle business logic such as user authentication, CRUD operations, and role-based access control.

Phase 5 — Frontend Development
The user interface is developed using HTML, CSS, and Bootstrap. Responsive pages are created for home, login, dashboards, and forms, ensuring accessibility across devices.

Phase 6 — Testing and Validation
The system is tested using various test cases to ensure proper functionality. Modules such as login, registration, data entry, and payment tracking are validated for accuracy and performance.

Phase 7 — Deployment
The application is deployed on a local server, making it accessible through a web browser. The system is designed to be scalable and can be hosted on cloud platforms for wider access.

1.6 Source of Data

The primary data used in the Online Gym Management System is generated and managed within the application itself. The system stores and processes data related to members, trainers, membership plans, and payments. All data is maintained in a relational database such as SQLite or MySQL, ensuring secure storage and efficient retrieval.

The data is collected through user inputs during registration, admin operations, and system interactions. This includes member details, trainer information, plan specifications, and payment records. The system dynamically updates and maintains records as users interact with the application.

1.7 Chapter Scheme

Chapter 1 — Introduction
Covers the background, motivation, literature review, objectives, significance, and research design of the Online Gym Management System.

Chapter 2 — Requirements Specification
Details the functional and non-functional requirements, hardware and software requirements, performance criteria, user characteristics, and system constraints.

Chapter 3 — System Design
Presents the system architecture, database design, data flow diagrams (DFD Level 0 and Level 1), activity diagram, flowchart, class diagram, ER diagram, and sequence diagram.

Chapter 4 — Implementation, Testing, and Maintenance
Explains the technologies used such as Python, Django, HTML, CSS, and Bootstrap. It also includes testing methods, test cases, validation techniques, and guidelines for system usage and maintenance.

Chapter 5 — Results and Discussion
Describes the system output, user interface screens (dashboards and forms), module-wise functionality, and database representation.

Chapter 6 — Summary and Conclusion
Summarizes the overall project, key features, and the outcomes achieved through the system.

Chapter 7 — Future Scope
Discusses possible improvements such as payment gateway integration, mobile application development, notification systems, and advanced analytics features.

.

Chapter 2: Requirements Specification

2.1 User Characteristics

The Online Gym Management System is designed for users with basic computer literacy, including gym administrators, trainers, and members. The target users are individuals who need to manage or access gym-related information efficiently through a web-based platform.

The users are expected to:

- Be able to operate a web browser on a desktop or mobile device
- Perform basic actions such as login, form submission, and navigation
- Understand simple user interface interactions such as clicking buttons, selecting options, and viewing dashboards

The system supports three types of users:

- **Admin:** Responsible for managing members, trainers, membership plans, and payments
- **Trainer:** Can view assigned members and monitor their details
- **Member:** Can register, log in, view personal details, and track payments

No advanced technical knowledge or programming skills are required. The system is designed to be user-friendly, with clear navigation, simple layouts, and easy-to-understand interfaces to ensure smooth interaction for all users.

2.2 Functional Requirements

1. **User Registration and Login**

The system shall allow users to register and log in securely using a username and password.

2. **Role-Based Access Control**

The system shall provide different access levels for Admin, Trainer, and Member, restricting functionalities based on user roles.

3. **Member Management**
The system shall allow the admin to add, update, view, and delete member details such as name, contact information, and assigned trainer.

4. **Trainer Management**
The system shall allow the admin to add, update, and delete trainer details and assign trainers to members.

5. **Membership Plan Management**
The system shall allow the admin to create, update, and manage different membership plans with details like price and duration.

6. **Payment Management**
The system shall record and manage payments made by members, including amount, date, and selected plan.

7. **Dashboard Access**
The system shall provide dashboards for Admin, Trainer, and Member with relevant information and summaries.

8. **Profile Viewing**
The system shall allow users to view their personal profile details and related information.

9. **Data Storage and Retrieval**
The system shall store all records securely in a database and allow quick retrieval when required.

10. **Logout Functionality**
The system shall allow users to securely log out of the application..

2.3 Dependencies

- Python 3.x
- Django Framework

● SQLite / MySQL Database
● HTML, CSS, JavaScript
● Bootstrap (for UI design)
● Django ORM
● Web Browser (Chrome / Edge / Firefox)
● IDE (PyCharm / VS Code

2.4 Performance Requirements

Response Time
• The system shall load pages and respond to user requests within 1–2 seconds under normal conditions.
• Actions such as login, data submission, and dashboard loading should occur with minimal delay.

Data Accuracy and Integrity
• The system shall maintain accurate records for members, trainers, plans, and payments.
• Data entered by users shall be validated to prevent errors and inconsistencies.

Throughput and Load
• The system shall support multiple users accessing the application simultaneously without significant performance degradation.
• It shall handle repeated operations such as registrations, updates, and payments efficiently.

Resource Usage
• The system shall run efficiently on standard hardware with at least 4 GB RAM and a basic CPU.
• It should not consume excessive memory or processing power, allowing smooth multitasking.

Stability and Reliability

• The system shall operate continuously without crashes or failures under normal usage conditions.
• It shall ensure reliable data storage and retrieval with minimal downtime.

User Experience
• The system shall provide a responsive and user-friendly interface with clear navigation.
• Feedback messages (e.g., successful login, data saved) shall be displayed instantly to users.
• The system shall ensure smooth interaction across both desktop and mobile devices.

2.5 Hardware Requirements

● Processor: Intel i3 or higher

● RAM: Minimum 4GB (8GB recommended)

● Storage: At least 500MB free space

● Operating System: Windows / Linux / macOS

● Internet Connection (for deployment and access)

2.6 Constraints & Assumptions

Constraints:

The system is designed for basic gym

operations and may not support advanced

features like real-time attendance tracking or

biometric authentication.

- The application runs on a local server during development and requires Django to be running on the host machine.

- Internet connection is not required after setup unless deployed online.
- The system does not include integrated online payment gateways (can be added in future scope).
- Performance may vary with a very large number of users or records if not optimized or deployed on scalable infrastructure.

- **Assumptions:**

- Users (Admin, Trainer, Member) have basic knowledge of using web applications.
- All data entered by users is assumed to be correct and valid.
- The system will be used in a controlled environment such as a gym or fitness center.
- The application will be accessed through modern web browsers on desktop or mobile devices.
- Proper authentication and authorization will be maintained for secure access to the system

Chapter 3: Design

3.1 Algorithm

Web-Based Gym Management System (Django MVT Architecture)

The core working of the Online Gym Management System is based on the Django Model-View-Template (MVT) architecture, which handles user requests, processes data, and displays results efficiently.

.

Step 1 — User Request (Input Handling)
Input: User actions such as login, registration, form submission

1. User sends request through web browser (URL or form)
2. Django URL dispatcher maps the request to the appropriate view
3. Request is validated (authentication, form validation)

Output: Validated request passed to the view

Step 2 — Business Logic Processing (Views)

1. Views process the request based on user role (Admin, Trainer, Member)
2. Perform operations such as:
    - Create (Add Member, Trainer, Plan)
    - Read (View dashboards, records)
    - Update (Edit details)
    - Delete (Remove records)
3. Apply role-based access control

Output: Processed data ready for database interaction

Step 3 — Data Management (Models & Database)

1. Models interact with the database using Django ORM
2. Data is stored/retrieved from tables such as:
    - Member
    - Trainer
    - Plan
    - Payment
3. Ensure data integrity and relationships

Output: Structured data returned to views

Step 4 — Response Generation (Templates)

1. Views pass data to templates
2. Templates (HTML + CSS + Bootstrap) render dynamic content
3. Generate user interface (dashboard, tables, forms)

Output: Web page displayed to user

Step 5 — Authentication & Authorization Logic

1. User login credentials are verified

2. Role-based redirection:
    - Admin → Admin Dashboard
    - Trainer → Trainer Dashboard
    - Member → Member Dashboard
3. Unauthorized access is restricted using @login_required

Step 6 — System Output

- Display updated dashboards
- Show confirmation messages (success/error)
- Reflect real-time updates in database

3.2 Function Oriented Design

The Online Gym Management System follows a modular, function-oriented design divided into the following components:

Authentication Module (register(), login_user(), logout_user())

- **Input:** User credentials (username, password)
- **Process:** Validate user, create account, authenticate login
- **Output:** User session created and redirected to respective dashboard

Member Management Module (add_member(), view_members(), update_member(), delete_member())

- **Input:** Member details (name, phone, plan, trainer)
- **Process:** Perform CRUD operations on member data
- **Output:** Updated member records stored in database

Trainer Management Module (add_trainer(), view_trainers(), update_trainer(), delete_trainer())

- **Input:** Trainer details (name, experience, contact)
- **Process:** Manage trainer records and assignments
- **Output:** Updated trainer data

Payment Module (make_payment(), view_payments())

- **Input:** Payment details (member, plan, amount)
- **Process:** Record transactions and update membership status
- **Output:** Payment history stored in database

Views Module (views.py)

- Handles all business logic and user requests
- Controls flow between models and templates
- Implements role-based dashboard logic

URL Routing Module (urls.py)

● Maps URLs to corresponding view functions
● Example: /login/, /register/, /admin_dashboard/

Database Module (models.py)

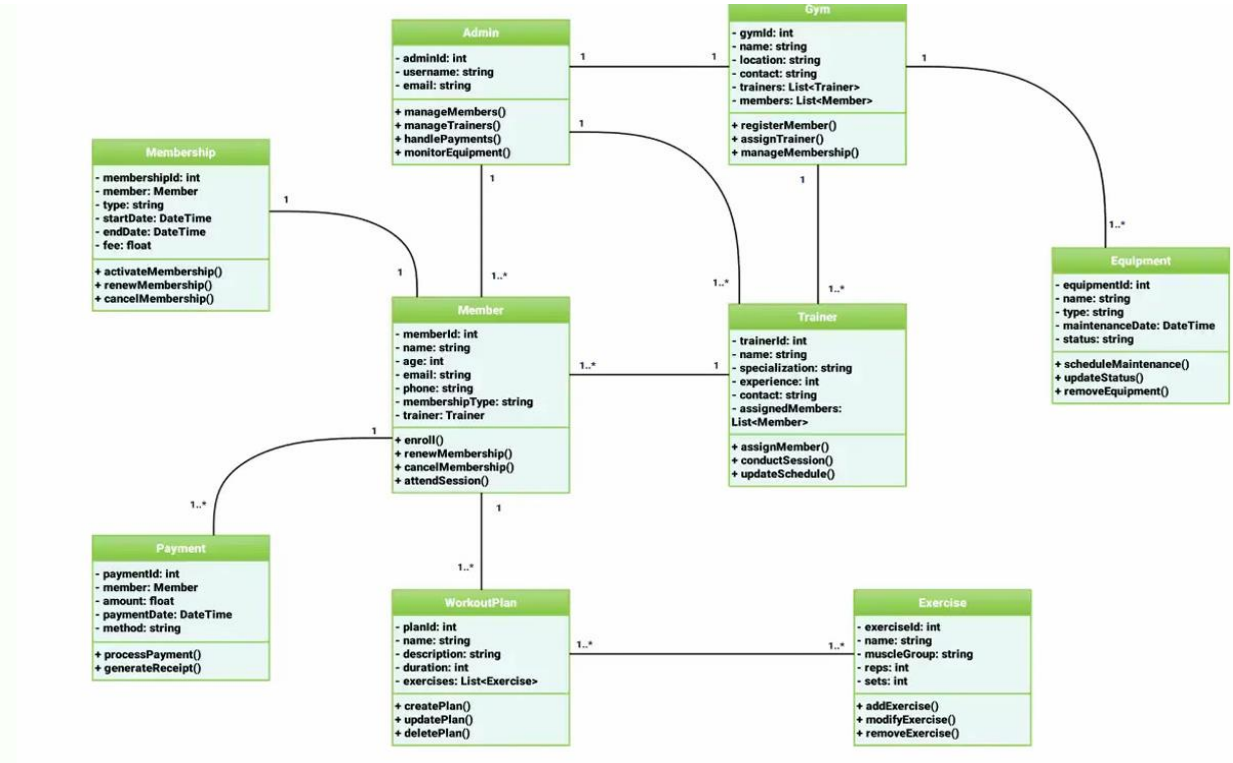
● Defines database structure: Member, Trainer, Plan, Payment
● Uses Django ORM for data handling

Template Module (HTML Files)

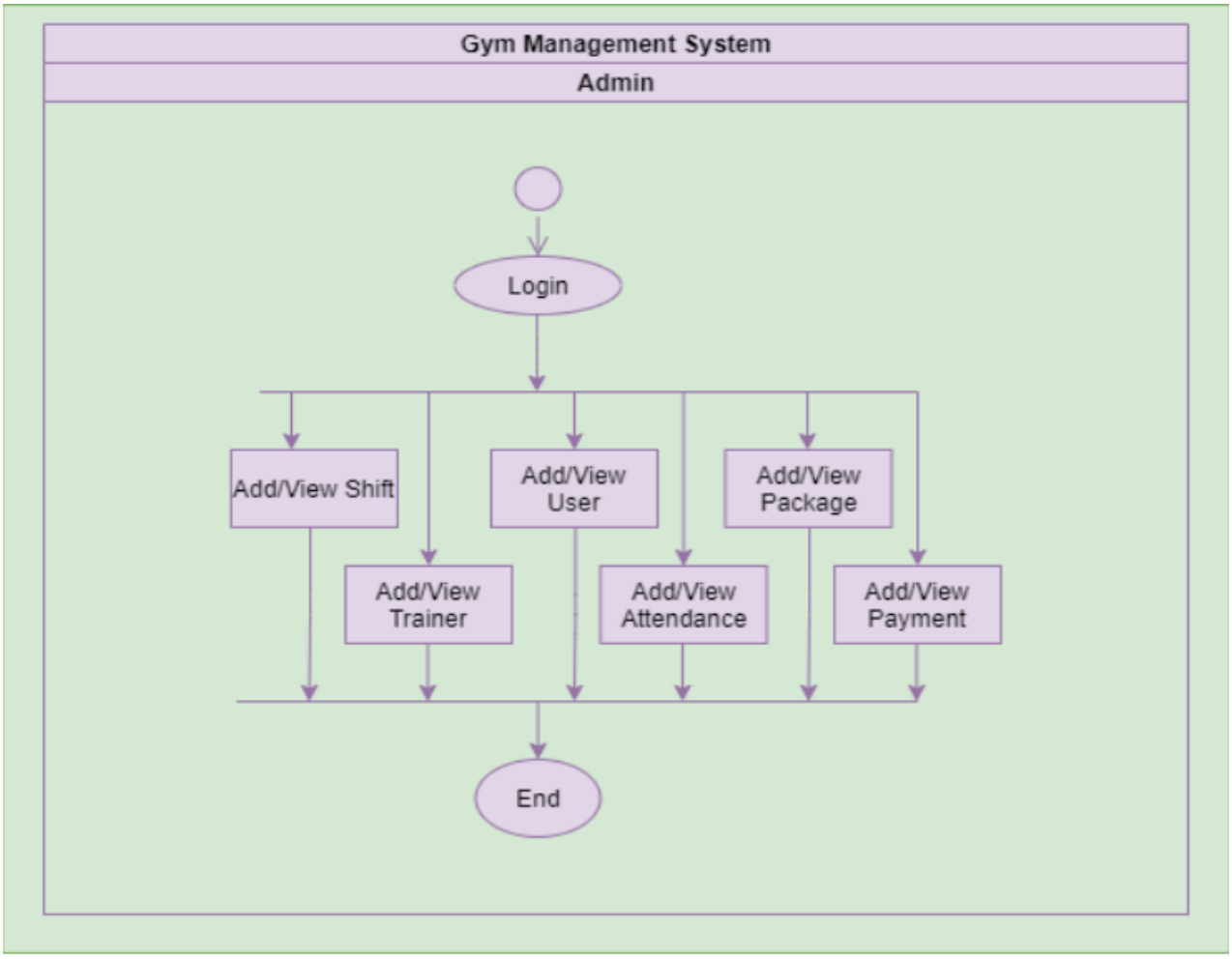
● Renders user interface (home, dashboards, forms)
● Displays dynamic data using Django template engine

3.3 System Design

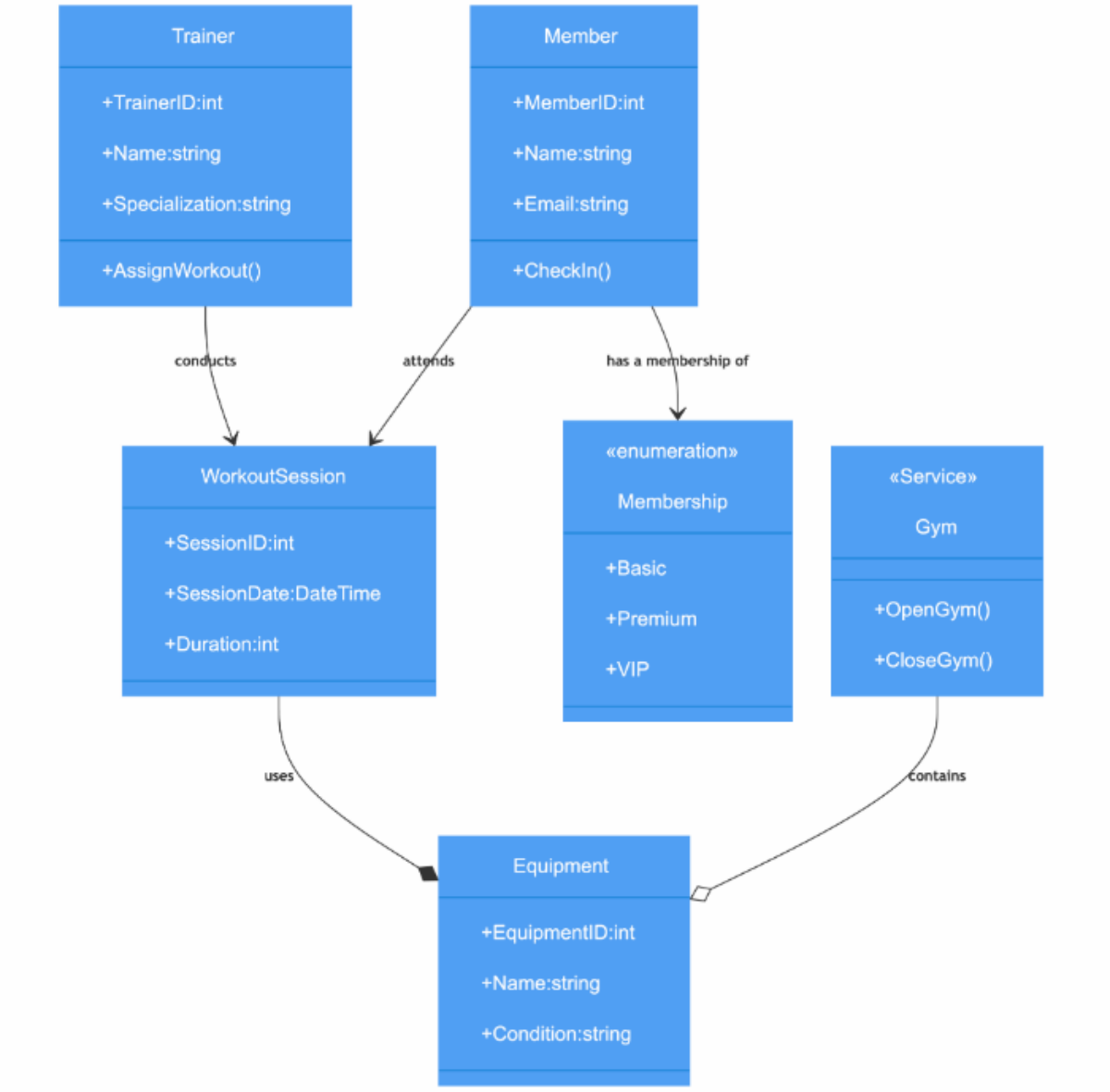
3.3.1 Data Flow Diagram



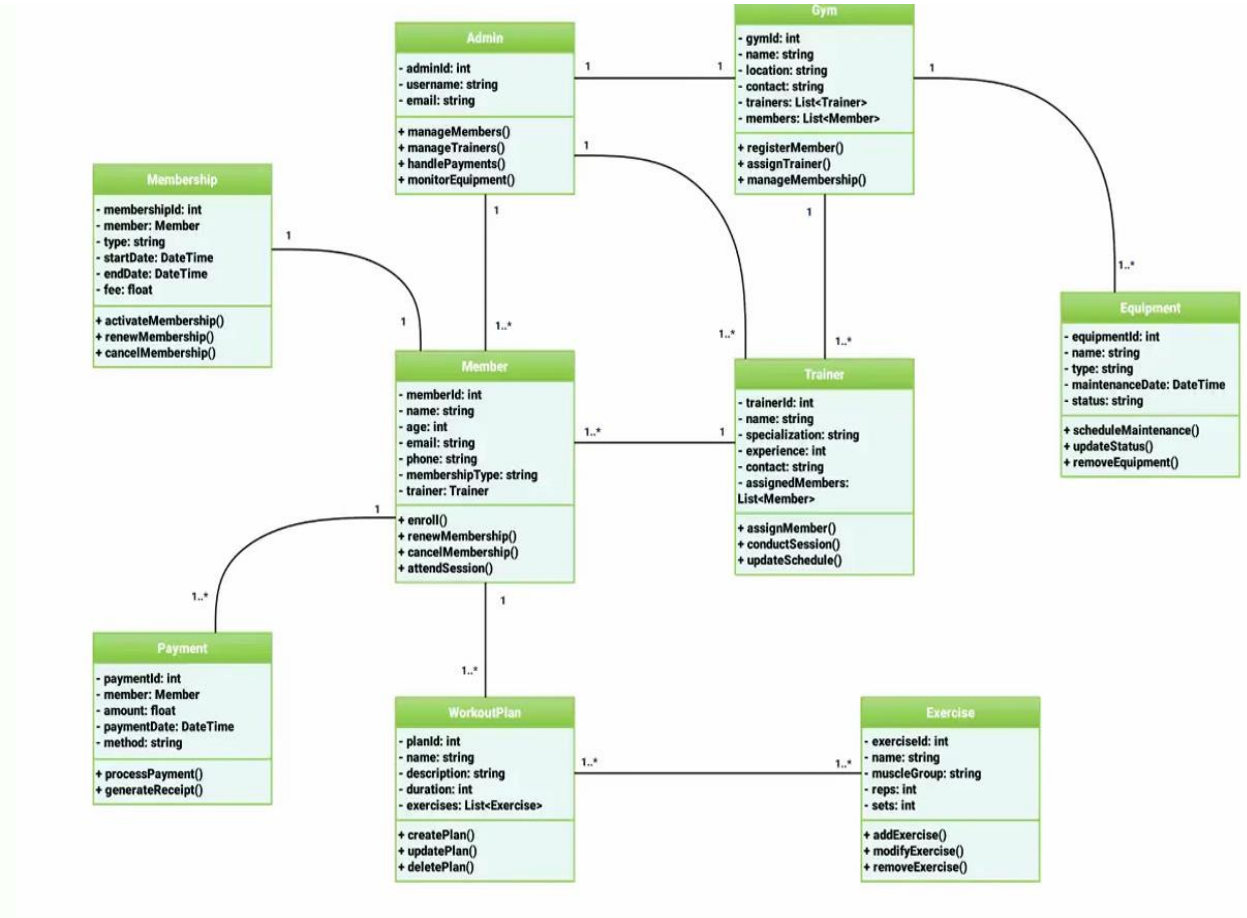
3.3.2 Activity Diagram



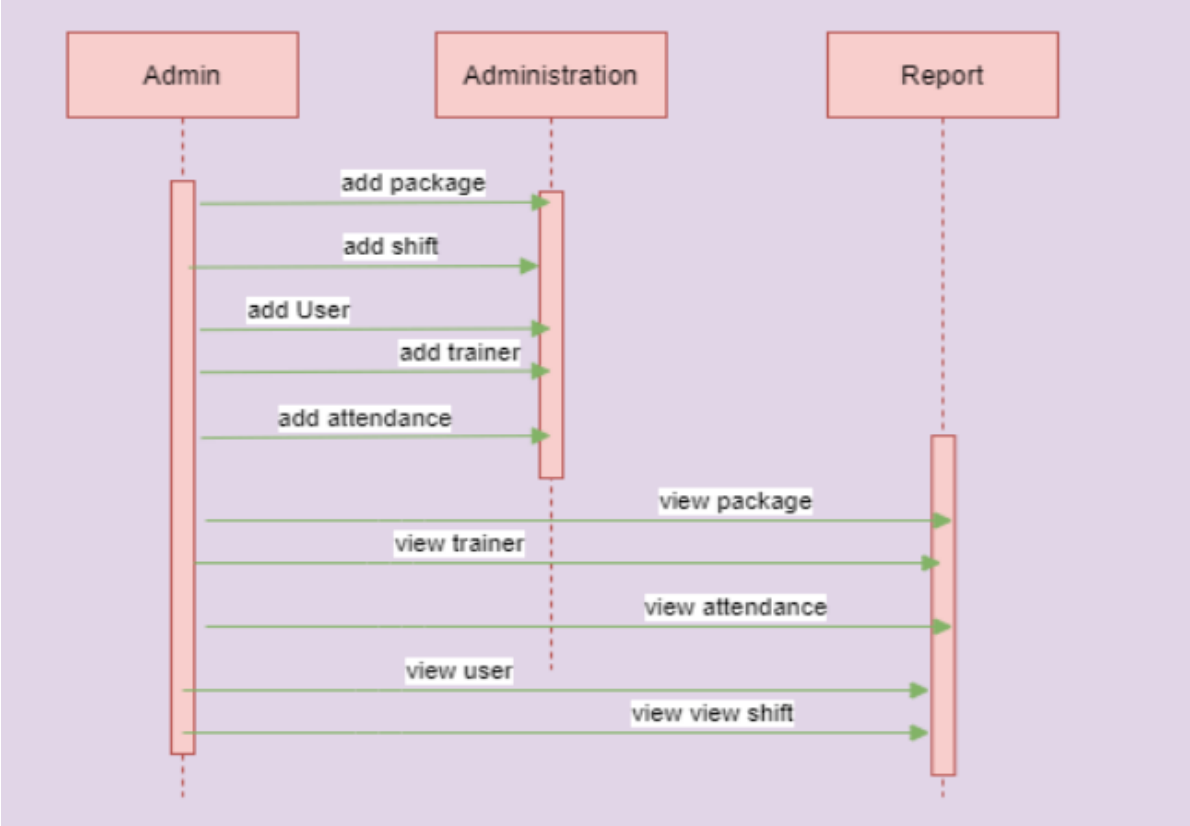
3.3.1 Class Diagram



3.3.2 **ER Diagram**



3.3.3 Sequence Diagram



3.4 Database Design

The Online Gym Management System uses a relational database for storing and managing all application data in a structured and efficient manner. Unlike file-based storage systems, this approach ensures data integrity, easy retrieval, and scalability. The system is implemented using SQLite (for development) or MySQL (for production) and interacts with the database through Django's built-in Object Relational Mapper (ORM).

All data is organized into well-defined tables representing different entities in the system. The main tables include:

- **User Table:** Stores authentication details such as username and password
- **Member Table:** Stores member information such as name, contact details, and assigned trainer
- **Trainer Table:** Stores trainer details including experience and contact information
- **Plan Table:** Contains membership plans with attributes like name, price, and duration
- **Payment Table:** Stores transaction details such as member, plan, amount, and payment date

Each table is linked through relationships (foreign keys), ensuring proper data association. For example, a member is linked to a trainer and a plan, and payments are linked to members and their selected plans.
The database operations such as Create, Read, Update, and Delete (CRUD) are handled using Django ORM, eliminating the need for writing complex SQL queries. This improves development speed and reduces the chances of errors.
The database ensures data consistency, security, and reliability by enforcing constraints and supporting authentication mechanisms. Since the system is web-based, all data is stored centrally and can be accessed securely by authorized users.
This design makes the system scalable, easy to maintain, and suitable for real-world gym management applications.

Chapter 4: Implementation, Testing, and Maintenance

4.1 Languages, IDEs, Tools and Technologies

The Online Gym Management System is implemented using modern web development technologies, programming languages, and tools that support efficient development, testing, and deployment of a scalable web application.

Programming Language

The system is developed using **Python (version 3.x)**, chosen for its simplicity, readability, and strong support for web development frameworks. Python enables rapid development and easy integration of backend functionalities such as authentication, database handling, and business logic.

1. Integrated Development Environment (IDE)

The project is developed and tested using **Visual Studio Code (VS Code)** and **PyCharm**, which provide features such as syntax highlighting, debugging tools, code completion, and project management. These IDEs help improve productivity and maintain clean code structure.

2. Web Framework

The system uses the **Django framework**, a high-level Python web framework that follows the Model-View-Template (MVT) architecture. Django provides built-in features such as authentication, URL routing, and ORM, making development faster and more secure.

3. Frontend Technologies

The user interface is developed using:

- **HTML** for structure

- **CSS** for styling
- **JavaScript** for basic interactivity
- **Bootstrap** for responsive design and modern UI components

These technologies ensure a clean, user-friendly, and mobile-responsive interface.

4. Database Technologies

The system uses **SQLite** for development and can be extended to **MySQL** for production environments. Django ORM is used to interact with the database, allowing efficient data storage and retrieval without writing complex SQL queries.

4.2 Testing Techniques and Test Plans

The Online Gym Management System is tested using functional, performance, usability, and robustness testing to ensure correctness and reliability. Functional testing verifies that all modules such as registration, login, member management, trainer management, plan creation, and payment tracking work as expected. Performance testing checks system response time, ensuring pages load quickly and operations are executed efficiently. Usability testing ensures the interface is user-friendly, easy to navigate, and provides clear feedback messages. Robustness testing evaluates system stability under multiple users and repeated operations.

The testing plan focuses on verifying system workflows, measuring response time, and ensuring consistent performance. The system is tested on a standard computer using a web browser with real user inputs such as registrations, logins, and data updates. Test cases include valid and invalid inputs to check error handling. The expected outcome is a stable, efficient, and user-friendly system that performs all operations accurately with minimal delay.

4.3 End User Instructions

1. Open a web browser and go to http://127.0.0.1:8000
2. On the home page, click on Register (for new users) or Login (for existing users)
3. Enter your credentials and log in to the system
4. Based on your role, access the respective dashboard (Admin, Trainer, or Member)
5. Admin can add/manage members, trainers, plans, and payments
6. Members can view their profile, plan details, and payment history
7. Trainers can view assigned members
8. Use the navigation menu to perform required actions and click Logout after use

Chapter 5: Results and Discussions

5.1 User Interface Representation

Home Page (/) — A clean and modern landing page with navigation options such as Home, Login, and Register. It provides a brief introduction to the system and allows users to access authentication pages.

Authentication Pages (/login, /register) — Simple and centered forms for user login and registration with input validation and user-friendly design.

Admin Dashboard (/admin_dashboard) — A dashboard with sidebar or navigation menu allowing the admin to manage members, trainers, membership plans, and payments. It displays data in tables with options to add, update, and delete records.

Member Dashboard (/member_dashboard) — Displays member profile details, assigned plan, and payment history in a structured format with easy navigation.

Trainer Dashboard (/trainer_dashboard) — Shows trainer profile and a list of assigned members.

Form Pages (/add_member, /add_trainer, /add_plan, /payment) — Clean and responsive forms for adding and managing data with labeled input fields and submit buttons

.

5.2 Brief Description of Modules

Authentication Module
This module handles user registration, login, and logout functionality. It verifies user credentials and provides secure access to the system. Based on the user role, it redirects users to their respective dashboards.

Admin Control Module
This module acts as the main control center for the system. The admin can manage all operations including adding, updating, and deleting members, trainers, membership plans, and payment records. It provides complete access to system data.

Member Management Module
This module manages all member-related information. It allows the admin to add new members, update their details, assign trainers, and track membership information efficiently.

Trainer Management Module
This module is used to manage trainer details such as experience, contact information, and assigned members. It helps maintain proper coordination between trainers and members.

Plan Management Module
This module allows the admin to create and manage different membership plans, including details like plan name, duration, and price. It helps in organizing gym subscriptions.

Payment Management Module
This module handles all payment-related operations. It records member payments, maintains transaction history, and helps track membership validity.

Dashboard Module
This module provides role-based dashboards for Admin, Trainer, and Member. It

displays relevant information such as summaries, lists, and records in a structured format.

System Monitoring Module
This module ensures smooth functioning of the system by maintaining data consistency and providing feedback messages such as successful operations or errors, helping users understand system status easily.

5.3 Snapshots with Brief Details

User Interface (UI) Design and Visual Description

1. Home Page (Landing Page)
The landing page features a clean and modern design with a dark theme and green accents. It includes a navigation bar with options like Home, Login, and Register. A central section provides a brief introduction to the system with call-to-action buttons for easy navigation.

2. Authentication Pages (Login & Register)
These pages have a simple and centered card layout with input fields for user credentials. The design is minimal, user-friendly, and includes validation messages for incorrect inputs.

3. Admin Dashboard (Main Workspace)
The admin dashboard provides a structured interface with navigation options to manage members, trainers, plans, and payments. Data is displayed in tables with action buttons for adding, updating, and deleting records. Summary sections may show key information like total members and revenue.

4. Member Dashboard
The member interface displays personal details, membership plan information, and payment history in a clean card-based layout. It allows easy access to relevant information in one place.

5. Trainer Dashboard
The trainer dashboard shows trainer details and a list of assigned members in an organized format, helping trainers track their responsibilities.

6. Form Pages (Add Member, Trainer, Plan, Payment)
These pages contain responsive forms with labeled input fields and submit buttons, ensuring easy data entry and validation.

5.4 Backend Representation

The system uses a relational database (SQLite/MySQL) to store all data related to users, members, trainers, plans, and payments. The database is managed using Django ORM, which handles all data operations such as insertion, retrieval, updating, and deletion. The application runs on a Django server and processes requests dynamically through views and templates. All user actions are reflected in the database in real time, ensuring data consistency and reliability

Chapter 6: Summary and Conclusions

Summary:

The Online Gym Management System successfully demonstrates a web-based solution for managing gym operations efficiently using the Django framework. The system automates key functionalities such as member registration, trainer management, membership plans, and payment tracking. It provides role-based access for Admin, Trainer, and Member, ensuring secure and organized data handling. The application features a responsive user interface accessible on both desktop and mobile devices, with a relational database (SQLite/MySQL) storing all records in real time..

Conclusion:

The project proves that a well-designed web application using Django, combined with a structured database and user-friendly interface, can effectively replace manual gym management processes. The system improves accuracy, reduces administrative workload, and enhances user experience without requiring complex infrastructure. The Online Gym Management System serves as a practical and scalable solution for modern fitness centers and can be further extended with advanced features for real-world deployment.

.

Chapter 7: Future Scope

1. Online Payment Integration — integrate payment gateways (Razorpay/Stripe) for secure online transactions

2. Mobile Application — develop a native Android/iOS app for better accessibility

3. Attendance Tracking — add feature to track member attendance and check-ins

4. Notification System — implement email/SMS alerts for membership expiry and payments

5. Advanced Analytics — provide reports on revenue, member growth, and gym performance

6. Multi-Branch Support — extend system to manage multiple gym branches

7. Role Expansion — add more roles like receptionist or manager with custom permissions

8. Cloud Deployment — deploy system on cloud platforms for global access

Appendix

```
GYM_Management_System/
├── manage.py
├── GYM/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
├── myg/
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── views.py
│   ├── urls.py
│   ├── forms.py
│   ├── migrations/
│   │   └── __init__.py
│   ├── templates/
```

```
│   │   ├── home.html
│   │   ├── login.html
│   │   ├── register.html
│   │   ├── admin_dashboard.html
│   │   ├── member_dashboard.html
│   │   ├── trainer_dashboard.html
│   │   ├── add_member.html
│   │   ├── add_trainer.html
│   │   ├── add_plan.html
│   │   └── payment.html
│   └── static/
│       ├── css/
│       │   └── style.css
│       ├── js/
│       │   └── script.js
│       └── images/
├── db.sqlite3
└── requirements.txt
```

Appendix B — Requirements

django>=4.0
sqlite3
mysqlclient (optional, for MySQL)
bootstrap>=5.0
pillow>=10.0
django-crispy-forms (optional)
python-dotenv (optional)

Appendix C — Run Instructions

pip install -r requirements.txt

python manage.py makemigrations

python manage.py migrate

python manage.py createsuperuser # optional (for admin access)

python manage.py runserver

Bibliography

1. Django Software Foundation, "Django Documentation," https://docs.djangoproject.com/, 2024.
2. Holovaty, A., and Kaplan-Moss, J., "The Definitive Guide to Django," Apress, 2009.
3. Python Software Foundation, "Python Documentation," https://docs.python.org/, 2024.
4. Bootstrap Team, "Bootstrap Documentation," https://getbootstrap.com/, 2024.
5. SQLite Consortium, "SQLite Documentation," https://www.sqlite.org/, 2024.
6. MySQL, "MySQL Documentation," https://dev.mysql.com/doc/, 2024.
7. Mozilla Developer Network (MDN), "HTML, CSS, and JavaScript Documentation," https://developer.mozilla.org/, 2024.
8. Sommerville, I., "Software Engineering," 10th Edition, Pearson, 2015.
9. Pressman, R. S., "Software Engineering: A Practitioner's Approach," McGraw-Hill, 2014.
10. W3Schools, "Web Development Tutorials," https://www.w3schools.com/, 2024.